

TechCorp Database Schema Guide

PostgreSQL Standards & Architecture Reference

Version: v3.4

Document: TC-DB-002

TechCorp Internal

1. Database Architecture

TechCorp uses a multi-region PostgreSQL architecture hosted on AWS RDS Aurora. The primary cluster resides in us-east-1 with read replicas in eu-west-1 and ap-southeast-1. All application services communicate via connection pooling through PgBouncer.

Engine	PostgreSQL 16.2 (Aurora-compatible)
Cloud Provider	AWS RDS Aurora – Multi-AZ
Regions	us-east-1 (primary), eu-west-1, ap-southeast-1 (read replicas)
Pooling	PgBouncer 1.22 — transaction mode, max 500 client connections
Migrations	Flyway 10.x — version-controlled, CI-gated
ORM	SQLAlchemy 2.x (Python), Prisma (Node.js)

2. PostgreSQL Standards

- Use PostgreSQL 16+ features: JSONB, generated columns, partial indexes, logical replication
- All tables must have explicit schema prefix (e.g., public.users, analytics.events)
- ENUM types must be defined as PostgreSQL custom types, not VARCHAR with CHECKs
- All monetary values stored as INTEGER (cents) — never FLOAT or NUMERIC for currency
- Timestamps always stored as TIMESTAMPTZ (timezone-aware UTC)
- Text search columns must use pg_trgm extension with GIN index

3. Naming Conventions

Entity	Convention	Example
Tables	snake_case, plural	user_accounts, api_keys
Columns	snake_case	created_at, tenant_id
Primary Keys	id (BIGSERIAL)	id BIGSERIAL PRIMARY KEY
Foreign Keys	{table}_id	tenant_id, user_id

Indexes	idx_{table}_{col}	idx_users_email
Unique Constraints	uq_{table}_{col}	uq_users_email
Check Constraints	ck_{table}_{rule}	ck_orders_status
Sequences	{table}_{col}_seq	users_id_seq
Views	vw_{name}	vw_active_subscriptions

4. Primary Keys

All tables use surrogate primary keys. New tables must use BIGSERIAL or UUIDv7 depending on whether external exposure is required.

Internal tables	BIGSERIAL — sequential, compact, efficient for joins
External-facing	UUID v7 — time-ordered, no enumeration risk
Composite PKs	Allowed only on pure join tables (no surrogate needed)
Example (BIGINT)	id BIGSERIAL PRIMARY KEY
Example (UUID)	id UUID DEFAULT gen_random_uuid() PRIMARY KEY

5. Foreign Keys

- All FK relationships must be declared explicitly — no implicit references
- Use ON DELETE RESTRICT by default; only ON DELETE CASCADE when parent deletion should propagate
- ON DELETE SET NULL only for optional/soft relationships
- Always index FK columns: CREATE INDEX idx_{child}_{fk_col} ON {child}({fk_col})
- Cross-schema FKs are forbidden — use application-level joins instead

6. Indexing Rules

Index Type	When to Use	Example
B-tree	Equality, range, ORDER BY	Most lookups
GIN	JSONB, array containment, full-text	tags @> ARRAY[...]
GiST	Geometric, range types, fuzzy text	tsvector columns
BRIN	Very large append-only time-series tables	created_at ranges
Partial	Filter frequently on a fixed condition	WHERE deleted_at IS NULL
Covering	Avoid heap fetch on hot queries	INCLUDE (col1, col2)

7. Backup Policy

Automated Backups	AWS RDS automated snapshots — retained 35 days
PITR	Point-in-Time Recovery enabled — granularity: 5 minutes
Manual Snapshots	Triggered before every major migration; retained 90 days
Cross-Region Copy	Daily snapshots copied to eu-west-1 within 1 hour
Restore Testing	Quarterly DR drill — restore must complete within RTO of 2h
Encrypted	AES-256 at rest, TLS 1.3 in transit

8. Example Tables

users — Core user identity table

```
CREATE TABLE public.users ( id BIGSERIAL PRIMARY KEY, uuid UUID NOT NULL DEFAULT
gen_random_uuid() UNIQUE, tenant_id BIGINT NOT NULL REFERENCES tenants(id) ON DELETE
RESTRICT, email TEXT NOT NULL, role user_role NOT NULL DEFAULT 'member', created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at TIMESTAMPTZ ); CREATE UNIQUE INDEX uq_users_tenant_email ON
users(tenant_id, email) WHERE deleted_at IS NULL; CREATE INDEX idx_users_tenant_id ON
users(tenant_id);
```

9. ER Diagram (Text-Based)

```
tenants (1) ■■< users (N) tenants (1) ■■< api_keys (N) users (1) ■■< sessions (N)
users (1) ■■< audit_logs (N) tenants (1) ■■< subscriptions (1) subscriptions (N)
>■■ plans (1) users (N) >■■< roles (M) [via user_roles join table] documents (N)
>■■ tenants (1) documents (1) ■■< document_chunks (N) document_chunks (N) >■■
embeddings (1)
```